Адаптивный ИИ-ассистент для обучения D&D

План проекта: Telegram-бот, курсовая работа и пет-проект

О проекте

Telegram-бот, который учит новичков играть в Dungeons & Dragons 5e. Бот выясняет уровень пользователя, отвечает на вопросы по официальным правилам (SRD 5.1), ведёт по темам от простого к сложному и проверяет знания квизами.

Стек

- **Бот** — Python, aiogram 3
- **Backend** — FastAPI
- **База данных** — PostgreSQL + pgvector, SQLAlchemy 2.0, Alembic
- **ИИ** — Claude Haiku или GigaChat; эмбеддинги через sentence-transformers
- **Инфраструктура** — Docker, docker-compose, Git / GitHub

Архитектура

```
Пользователь → Telegram → bot (aiogram)
                                ↓
                          api (FastAPI) ──→ LLM
                                ↓
                      PostgreSQL + pgvector
```

```
docker compose
  ├── db   – PostgreSQL + pgvector
  ├── api  – FastAPI: вся логика (пользователи, LLM, RAG, квизы)
  └── bot  – aiogram: только общение с Telegram
```

Этап 0. Подготовка

1. Репозиторий на GitHub, `.gitignore`, `.env` (ключи не попадают в git).
2. Виртуальное окружение и `requirements.txt`.
3. Токен бота у @BotFather.
4. Ключ к LLM.
5. Установить Docker Desktop, проверить командой `docker run hello-world`.

Этап 1. Первые «оживления»

6. Скрипт `test_llm.py`: вопрос → ответ модели.
7. Эхо-бот на aiogram.
8. Бот отвечает через LLM (пока напрямую).

✓ **Результат: бот-«ChatGPT» в Telegram.**

Этап 2. FastAPI

9. Hello-world: `GET /health`, документация на `/docs`.
10. `POST /ask`: логика LLM переезжает в API.
11. Бот обращается к API через `httpx`.
12. Системный промпт наставника по D&D.

✓ **Результат: работает схема «бот → API → LLM».**

Этап 3. Docker и PostgreSQL

13. `docker-compose.yml` с PostgreSQL.
14. SQLAlchemy + Alembic: модель `User`, первая миграция.
15. Эндпоинт «получить или создать пользователя по `telegram_id`».
16. Dockerfile для api и bot; весь проект поднимается через `docker compose up`.

✓ **Результат: проект целиком в Docker.**

Этап 4. Знакомство и память

17. Кнопки и онбординг через FSM.
18. Ответы онбординга сохраняются в БД.
19. Уровень пользователя подставляется в промпт.
20. Таблица `messages`: история диалога, последние 10 сообщений идут в промпт.
21. Индикатор «печатает…».

✓ **Результат: бот знает пользователя и подстраивается под него.**

Этап 5. RAG — поиск по правилам (самый сложный)

22. Образ PostgreSQL заменить на `pgvector/pgvector`, включить расширение.
23. Скачать SRD 5.1, написать скрипт нарезки на куски.
24. Посчитать эмбеддинги и сохранить в таблицу `rule_chunks`.
25. Векторный поиск: 3 подходящих куска на вопрос.
26. Правила в промпте `/ask`, ссылка на источник в ответе.

✓ **Результат: ответы по официальным правилам, без выдумок.**

Этап 6. Адаптивное обучение

27. Таблицы `concepts`, `concept_prerequisites`, `user_concepts`; заполнить 10–15 тем.
28. `/progress`: прогресс полосками.

29. `/quiz`: вопросы с кнопками, обновление уровня `mastery`.
30. `/learn`: выбор следующей темы по пререквизитам.
31. Повторение давно не повторявшихся тем.

✓ **Результат: MVP для курсовой готов.**

Этап 7. Доводка

32. Обработка ошибок (LLM не ответил, странный ввод).
33. Лимит запросов на пользователя в день.
34. Тесты на pytest для API.
35. Тестовый набор из 20–30 вопросов: сравнить точность с RAG и без.
36. README со скриншотами и инструкцией запуска.

✓ **Результат: проект готов к сдаче.**

Потом (пет-проект)

- Учебное приключение: ИИ-мастер, броски кубиков через tool use, HP и сцена в БД.
- Помощник по созданию персонажа.
- Деплой на VPS: бот работает 24/7.
- CI в GitHub Actions: тесты на каждый пуш.
- Кэширование промптов, разные модели под разные задачи.
- Веб-версия.

Сложность этапов

- **0–2. Подготовка, первые шаги, FastAPI** — лёгкие
- **3. Docker и PostgreSQL** — средний
- **4. Знакомство и память** — средний
- **5. RAG** — самый сложный
- **6. Адаптивное обучение** — средний, но объёмный
- **7. Доводка** — лёгкий

Таблицы базы данных

- `users` — telegram_id, имя, уровень, интересы, счётчик запросов
- `concepts` — темы для изучения (кубики, характеристики, бой…)
- `concept_prerequisites` — какие темы нужно знать до данной
- `user_concepts` — уровень знания темы (mastery 0–1), дата повторения
- `rule_chunks` — куски правил SRD и их векторы (embedding)
- `quiz_questions`, `quiz_answers` — вопросы квизов и история ответов
- `messages` — история диалога

- `characters`, `adventures` — персонаж и приключение (позже)

Стоимость ИИ (Claude)

- **Haiku 5.5** — ~$0.5 — ~$1–1.5
- **Sonnet 5.5** — ~$9 — ~$18–27

Оценка на запрос ~3000 токенов на входе и ~300 на выходе. Anthropic официально не обслуживает Россию; легальная альтернатива — GigaChat или YandexGPT. Вызов ИИ вынесен в один модуль `llm.py`, провайдер меняется в одном месте.